

PROJECT PROPOSAL
“IDENTIFIKASI TINGKAT KATA KASAR BERBAHASA INDONESIA
DALAM PLATFORM MEDIA SOSIAL BERBASIS TEKS”
NATURAL LANGUAGE PROCESSING

Group 10 - LA01

2802403183 - Brian Nicholas Tedjo

2802419446 - Jason Budiharjo

2802402275 - Marvin Adriano Rusdianto

BAB 1. PENDAHULUAN

1.1. Latar Belakang

Komunikasi melalui media sosial telah menjadi metode yang umum digunakan oleh masyarakat. Komunikasi menggunakan teks menjadi cara komunikasi yang paling banyak diterapkan dalam *platform* media sosial. Beberapa *platform* seperti X, Threads, dan Reddit menggunakan teks sebagai basis utama komunikasinya. Beberapa yang lain seperti Instagram, YouTube, dan TikTok juga menerapkan penggunaan teks dalam fitur komentar, walaupun bukan sebagai media utama dalam berkomunikasi. Saat ini pengguna media sosial didominasi oleh generasi Z. Penggunaan bahasa gaul di kalangan remaja merupakan hal yang tak bisa dihindari dari zaman ke zaman, termasuk di kalangan generasi Z. Dengan berkembangnya zaman kosakata gaul semakin bertambah, termasuk bertambahnya kosakata gaul yang bernuansa negatif. Banyaknya penggunaan bahasa gaul yang bernuansa negatif ini menjadikan kata kasar sebagai bahasa gaul yang sering digunakan. Oleh sebab itu tingginya komunikasi media sosial berbasis teks turut mendorong penyebaran kata-kata kasar melalui teks. Saat ini fenomena ini telah terjadi di seluruh dunia, termasuk Indonesia. Dengan perspektif tersebut, penelitian ini dilakukan untuk mengidentifikasi kata-kata kasar berbahasa Indonesia dalam platform media sosial yang berbasis teks.

1.2. Rumusan Masalah

Penelitian ini merumuskan permasalahan sebagai berikut:

1. Bagaimana cara mengidentifikasi kata-kata kasar dalam teks berbahasa Indonesia pada platform media sosial?
2. Bagaimana menangani variasi penulisan dan penyamaran kata kasar yang dilakukan pengguna?

3. Bagaimana membedakan kata yang bersifat kasar dan kata yang bermakna netral dalam konteks kalimat yang berbeda?

1.3. Tujuan

Penelitian ini memiliki beberapa tujuan utama sebagai berikut:

1. Mengidentifikasi kata-kata kasar pada teks;
2. Menerapkan Natural Language Processing dalam proses identifikasi;

BAB 2. PENJELASAN METODE

2.1. Model Yang Digunakan

Dalam penelitian ini digunakan dua pendekatan utama dalam klasifikasi teks, yaitu metode machine learning tradisional dan metode berbasis deep learning. Model yang digunakan dalam project ini adalah Naive Bayes dan *indoBERT*. Pemilihan kedua model ini bertujuan untuk membandingkan performa antara metode sederhana dan metode berbasis transformer dalam mendeteksi kata-kata kasar berbahasa Indonesia

2.1.1. Logistic Regression

Logistic Regression merupakan algoritma klasifikasi linear yang digunakan untuk memprediksi probabilitas suatu data termasuk ke dalam kelas tertentu. Dalam konteks klasifikasi teks, model ini bekerja dengan mempelajari hubungan antara fitur-fitur teks dengan label yang diberikan.

Pada penelitian ini, Logistic Regression digunakan dengan representasi fitur berupa TF-IDF, khususnya dengan pendekatan character n-gram. Pendekatan ini memungkinkan model untuk mengenali pola penulisan kata kasar, termasuk yang telah dimodifikasi atau disamarkan.

2.1.2. Naive Bayes

Naive Bayes merupakan algoritma klasifikasi berbasis probabilitas yang menggunakan Teorema Bayes dengan asumsi bahwa setiap fitur bersifat independen. Model ini banyak digunakan dalam klasifikasi teks karena memiliki keunggulan dalam hal kecepatan dan efisiensi dalam komputasi.

Dalam konteks penelitian ini, Naive Bayes mengklasifikasikan teks berdasarkan fitur yang diekstraksi menggunakan metode seperti *Bag of Words* atau *TF-IDF*. Model ini sering memberikan hasil yang cukup baik untuk permasalahan klasifikasi teks.

Model ini bekerja secara efektif dalam mengenali kata-kata kasar yang muncul secara eksplisit dalam teks, namun memiliki keterbatasan dalam memahami konteks kalimat secara keseluruhan.

2.1.3. IndoBERT

IndoBERT merupakan model bahasa berbasis arsitektur atau transformer yang dikembangkan khusus untuk bahasa Indonesia. Model ini merupakan adaptasi dari BERT (*Bidirectional Encoder Representations from Transformer*) yang mampu memahami konteks kata dalam kalimat secara dua arah.

IndoBERT memiliki keunggulan dalam menangkap hubungan antar kata dalam suatu kalimat, sehingga lebih efektif dalam memahami makna teks dibandingkan metode tradisional. Dalam penelitian ini, IndoBERT digunakan untuk melakukan klasifikasi tingkat kata kasar dengan memanfaatkan representasi embedding yang dihasilkan oleh model.

IndoBERT tidak menggunakan representasi TF-IDF, melainkan memanfaatkan teknik tokenisasi berbasis subword dan embedding kontekstual untuk memahami makna kalimat secara menyeluruh. Model ini sangat efektif dalam membedakan penggunaan kata yang memiliki makna ganda berdasarkan konteks.

2.2. Natural Language Processing (NLP)

Natural Language Processing adalah bidang keilmuan yang membuat mesin mampu untuk memahami, menginterpretasikan, dan menghasilkan bahasa manusia (LLC, 2025, 27). Sedangkan menurut Apriliani, dkk. (2023), *Natural Language Processing* adalah cabang dari *Artificial Intelligence* yang membantu komputer membaca dan memahami bahasa alami manusia. Penelitian ini melakukan identifikasi kata-kata kasar pada sebuah teks, penelitian yang dilakukan termasuk dalam bidang keilmuan *Natural Language Processing*.

2.3. Library dan Tools

2.3.1. NLTK

NLTK (*Natural Language Toolkit*) merupakan salah satu *library* Python yang digunakan untuk pemrosesan bahasa alami (*Natural Language Processing*). *Library* ini menyediakan berbagai fungsi untuk membantu pengolahan teks, seperti tokenisasi, penghapusan stopwords, serta analisis linguistik dasar. Dalam penelitian ini, NLTK digunakan terutama pada tahap preprocessing data teks. Fungsi utama NLTK yang dimanfaatkan adalah:

1. Tokenisasi

Tokenisasi adalah proses memecah kalimat menjadi unit-unit kecil berupa kata (token). Proses ini penting agar teks dapat dianalisis lebih lanjut oleh model.

Contoh: "Indonesia lagi macet parah" = ["Indonesia", "lagi", "macet", "parah"]

2. *Stopwords Removal*

NLTK menyediakan daftar kata umum (stopwords) yang sering muncul tetapi tidak memiliki makna signifikan seperti "dan", "yang", "di". Penghapusan stopwords bertujuan untuk mengurangi noise dalam data.

3. *Text Processing Support*

NLTK juga membantu dalam pengolahan teks dasar seperti normalisasi dan integrasi dengan pipeline lainnya.

2.3.2. PySastrawi

PySastrawi adalah sebuah *library* Python yang digunakan untuk membantu menghapus *stopwords* dan melakukan *stemming*. (Prahasto & Setiawan, 2025, 22) Pada penelitian ini PySastrawi digunakan pada tahap *text preprocessing*. PySastrawi digunakan untuk melakukan *stopword removal*, ini bertujuan untuk mengurangi dimensi data dengan menghapus kata yang tidak memiliki kontribusi terhadap sentimen, seperti kata ganti benda, konjungsi dan preposisi. Selain itu, pada penelitian ini PySastrawi digunakan untuk melakukan *stemming* terhadap data, yaitu mengonversi kata berimbuhan menjadi bentuk dasarnya.

2.3.3 Rapidfuzz

RapidFuzz merupakan salah satu *library* Python yang digunakan untuk melakukan fuzzy string matching, yaitu metode yang digunakan untuk mengukur tingkat kemiripan antara dua string meskipun tidak memiliki kesamaan secara persis. Teknik ini sangat berguna dalam pengolahan teks yang mengandung variasi penulisan atau perubahan karakter.

Dalam penelitian ini, RapidFuzz dimanfaatkan untuk mendeteksi kata-kata kasar yang telah mengalami modifikasi atau penyamaran. Pada platform media sosial, pengguna sering kali mengubah penulisan kata kasar dengan tujuan menghindari sistem moderasi, contohnya dengan mengganti huruf dengan angka, menambahkan karakter berulang, atau menghilangkan beberapa huruf dalam kata tersebut.

Metode *exact matching* tidak mampu menangani kasus seperti ini secara efektif. Oleh karena itu, RapidFuzz digunakan untuk menghitung tingkat kemiripan antara kata dalam teks dengan kata-kata dalam kamus kasar. Apabila nilai kemiripan melebihi ambang batas tertentu, maka kata tersebut dapat diidentifikasi sebagai bentuk variasi dari kata kasar yang dimaksud. Dengan demikian, penggunaan RapidFuzz dalam penelitian ini berperan sebagai pelengkap dalam tahap *preprocessing*, khususnya dalam meningkatkan kemampuan sistem untuk mengenali kata kasar yang tidak dituliskan secara baku.

2.4. AutoTokenizer (Hugging Face)

AutoTokenizer adalah komponen dari *library* Hugging Face Transformers yang digunakan untuk melakukan proses tokenisasi teks secara otomatis sesuai dengan model yang digunakan. Tokenisasi adalah proses mengubah teks menjadi token, yang kemudian akan diproses oleh model machine learning. Berbeda dengan metode tokenisasi tradisional, AutoTokenizer menggunakan pendekatan subword tokenization, yaitu memecah kata menjadi bagian-bagian yang lebih kecil berdasarkan struktur yang telah dipelajari sebelumnya oleh model. Pendekatan ini memungkinkan sistem untuk tetap memahami kata-kata yang tidak terdapat dalam kamus, termasuk kata tidak baku atau kaya yang telah dimodifikasi.

Dalam penelitian ini, AutoTokenizer digunakan bersama model IndoBERT untuk memastikan bahwa proses tokenisasi sesuai dengan format yang diharapkan oleh model tersebut. Hal ini penting karena setiap model berbasis *transformer* memiliki aturan tokenisasi yang berbeda, sehingga penggunaan tokenizer yang sesuai akan mempengaruhi performa model secara signifikan. AutoTokenizer juga menangani proses tambahan seperti pemberian indeks numerik pada token, penambahan token khusus, serta penyesuaian panjang input melalui *padding* dan *truncation*. AutoTokenizer berperan penting dalam menjembatani data teks mentah menjadi input yang dapat diproses oleh model IndoBERT.

2.5. Scikit-Learn (evaluate module)

Scikit-learn, yang sering disebut sklearn adalah sebuah *library open-source* berbasis bahasa pemrograman Python yang dirancang untuk menyederhanakan implementasi model Machine Learning. Scikit-learn memiliki modul penting yang sering digunakan dalam pembuatan Machine Learning seperti, klasifikasi, regresi, dan clustering. Scikit-learn dibangun menggunakan fondasi library Python lainnya, seperti NumPy

untuk pemrosesan *array*, SciPy untuk algoritma matematika tingkat lanjut, dan Matplotlib untuk visualisasi data. Penelitian ini menggunakan Scikit-learn untuk melakukan evaluasi model, scikit-learn menyediakan serangkaian metrik bawaan sehingga membantu dalam proses evaluasi model. Metrik evaluasi yang digunakan adalah *F1-score*, yaitu rata-rata harmonik dari presisi dan *recall*. Metrik *F1-score* dipilih karena memberikan keseimbangan antara metrik presisi dan *recall* ini.

BAB 3. Pengerjaan Proyek

3.1. Pengumpulan Dataset

Dataset yang digunakan untuk melatih model AI dibagi menjadi data primer dan sekunder. Data primer didapatkan melalui *web scraping* dari situs media sosial Reddit, Thread, dan X. *Scraping* menggunakan *library* Python Selenium dan menargetkan *post* yang berhubungan dengan Indonesia secara umum. Beberapa *tag* yang digunakan dalam pencarian meliputi “Timnas Indonesia”, “Politik Indonesia”, “Macet Jakarta”, “Harga naik”, “Makan bergizi gratis”, dan “lang:id”.

Data sekunder diambil dari dataset yang dipublikasikan secara *open source* pada Kaggle. Dataset sekunder ini terdiri atas kalimat-kalimat dalam bahasa Indonesia dari platform X (Twitter) yang diambil pada tahun 2019 dan dapat diakses melalui tautan: <https://www.kaggle.com/datasets/ilhamfp31/indonesian-abusive-and-hate-speech-twitter-text>. Dataset tersebut telah dilabeli dengan kolom-kolom berikut

1. HS: *hate speech label*;
2. Abusive: *abusive language label*;
3. HS_Individual: *hate speech targeted to an individual*;
4. HS_Group: *hate speech targeted to a group*;
5. HS_Religion: *hate speech related to religion/creed*;
6. HS_Race: *hate speech related to race/ethnicity*;
7. HS_Physical: *hate speech related to physical/disability*;
8. HS_Gender: *hate speech related to gender/sexual orientation*;
9. HS_Gender: *hate related to other invective/slander*;
10. HS_Weak: *weak hate speech*;
11. HS_Moderate: *moderate hate speech*;
12. HS_Strong: *strong hate speech*.

Data dari kedua sumber digabungkan dan dilabeli ulang menggunakan skema anotasi yang seragam. Data final disimpan dalam file .csv dengan kolom ["Kalimat Asli",

"Kalimat Dinormalisasi", "Level Kata Kasar", "Disamarkan"]. Label "Level Kata Kasar" dibagi menjadi empat level seperti dijelaskan pada Tabel 1 berikut.

Level	Tingkat Kekasaran	Deskripsi	Contoh kata
0	Nihil	Bukan kata kasar	-
1	Rendah	Sering digunakan dalam percakapan sehari-hari, dapat menimbulkan ketidaknyamanan ringan. Tidak menyerang pihak lain secara personal maupun kelompok, tidak eksplisit, dan tidak menyinggung ras atau agama. Kata pada level ini tidak memiliki makna ganda sebagai kata biasa sehingga aman untuk dimasukkan ke dalam leksikon.	Bego, goblok, tolol, idiot, bodoh, dungu, sinting, edan, gila, kampungan
2	Menengah	Menimbulkan ketidaknyamanan jika digunakan di ruang publik dan menyinggung pihak lain secara langsung, namun tidak eksplisit maupun menyinggung ras atau agama.	Anjing, bangsat, bajingan, brengsek, keparat, sialan, kurang ajar, babi, kampret, asu, celaka,
3	Tinggi	Menyerang individu/kelompok/ras lain, mengandung konten eksplisit	Jancok / jancuk, kontol, memek, asu, dancok, lonte, pelacur

Tabel 1. Penjelasan level tingkat kata kasar

Catatan penting mengenai Tabel 1: kata-kata seperti ‘gila’, ‘edan’, ‘anjing’, dan ‘babi’ tidak dimasukkan ke dalam level manapun pada leksikon karena memiliki makna ganda. Kata-kata tersebut ditangani secara kontekstual oleh model IndoBERT pada tahap klasifikasi (lihat Bagian 3.3.1 dan 3.3.2 untuk penjelasan lebih lanjut).

Label “Disamarkan” bermaksud untuk menandai kalimat yang mengandung kata kasar dengan disamarkan menggunakan kombinasi huruf, simbol, dan angka (contohnya: ‘anjing’ -> ‘4nj!ng’). Nilai 0 menyatakan bahwa tidak ada kata kasar yang disamarkan, sedangkan 1 menyatakan terdapat kata kasar yang disamarkan.

3.2. *Preprocessing*

Preprocessing atau pengolahan data dilakukan untuk memastikan data siap diproses lebih lanjut. Tahapan ini dibagi menjadi 7 tahapan dengan proses sebagai berikut:

3.2.1. **Deteksi Penyamaran**

Bagian ini menerapkan RegEx untuk mendeteksi adanya kata yang disamarkan menggunakan karakter non-alfanumerik dan angka pada data yang belum di *preprocess*. Berikut adalah *pattern* RegEx yang digunakan: `[0-9@!$.|]`. Kalimat yang mengandung karakter sesuai *pattern* RegEx tersebut kemudian akan dilabeli dengan angka 1 di kolom 'Disamarkan'.

3.2.2. **Lowercasing**

Menggunakan fungsi `.lower()` pada Python untuk memastikan seluruh kalimat yang diambil dari dataset memiliki format yang selaras.

3.2.3. **Penghapusan spasi berlebih**

Setiap spasi yang membentuk pola berulang (contohnya 'a n j i n g') akan dihapus untuk digabungkan menjadi satu kesatuan kata yang utuh. *Regular expression* digunakan sebagai berikut: `(?<!\w)(\w\s){2,}\w(?!\w)`.

3.2.4. **Pembersihan tanda baca**

Menggunakan *regular expression* dengan format `(?<=[a-z])[.\-_'";](?=[a-z])` untuk menghilangkan tanda baca seperti titik (.), strip (-), dan garis bawah (_), dan lainnya yang jarang muncul sebagai substitusi huruf dalam suatu kata kasar. Adapun tanda baca seperti asteris (*), seru (!), dan arroba (@) tidak dibersihkan karena sering muncul dalam kata kasar.

3.2.5. **Substitusi tanda baca dan angka**

Sisa tanda baca dan angka yang umum muncul sebagai substitusi dari huruf dikonversi kembali ke bentuk huruf.

3.2.6. **Tokenisasi**

Mengubah kalimat menjadi token-token yang berupa kata. Proses ini menggunakan tokenizer yang disediakan oleh Library Python PySastrawi.

3.2.7 ***Stemming***

Menghapus imbuhan pada suatu kata sehingga kembali ke bentuk dasarnya (misal: 'mengambil' -> 'ambil') agar kata-kata dengan makna sama digabungkan menjadi satu kesatuan. Tahapan ini dilakukan dengan bantuan Library Python 'PySastrawi'. Pada kasus ini, *stemming* digunakan ketimbang

lemmatizing karena kata dalam bahasa Indonesia tidak mengalami perubahan kata seperti pada bahasa Inggris, melainkan hanya mendapatkan imbuhan.

3.3. Feature Extraction

Data yang telah melewati tahap *preprocessing* kemudian diubah ke dalam bentuk fitur yang dapat dipahami oleh model. Tahap ini dilakukan menggunakan 2 metode utama, yakni Lexicon dan LLM-Assisted Labelling.

3.3.1. Lexicon Kata Kasar

Lexicon berisi daftar kata-kata kasar yang telah diberikan, beserta dengan tingkat kekasarannya. Dalam tabel lexicon tersebut, kata-kata dengan makna ganda tidak dimasukkan demi mencegah kesalahan konteks. Token diberi label berdasarkan kata-kata dalam tabel lexicon untuk mempercepat proses awal. Beberapa kata yang dimasukkan ke lexicon seperti ‘bangsat’, ‘tolol’, ‘kontol’, dan ‘bajingan’. Kata-kata dengan makna ganda seperti ‘anjing’, ‘kampung’, dan ‘jawa’ tidak dimasukkan ke dalam lexicon karena memiliki kemungkinan digunakan dalam konteks lain.

3.3.2. LLM-Assisted Labelling

Kata-kata yang belum mendapatkan label dari leksikon diolah menggunakan LLM-Assisted Labelling untuk memastikan seluruh data mendapatkan anotasi yang tepat. Dalam konteks ini, model bahasa besar (seperti GPT-4o-mini) digunakan untuk menangkap konteks yang terdapat dalam suatu kalimat sebelum diberikan label. Variabel tingkat kekasaran menggunakan Tabel 1.

3.3.3. Ekstraksi Fitur TF-IDF

Untuk model Naive Bayes dan Logistic Regression, teks yang telah diproses diubah menjadi representasi numerik menggunakan metode TF-IDF (Term Frequency-Inverse Document Frequency) dari scikit-learn. TF-IDF mengukur seberapa penting suatu kata dalam sebuah dokumen relatif terhadap seluruh korpus. Representasi ini digunakan sebagai input langsung ke kedua model tradisional tersebut. IndoBERT tidak menggunakan TF-IDF — model ini menerima token dari AutoTokenizer dan menghasilkan representasi embedding kontekstualnya sendiri.

3.4. Model

Pada proyek ini digunakan tiga pendekatan model, yaitu Logistic Regression dan Naive Bayes sebagai machine learning tradisional dan IndoBERT sebagai deep learning.

Naive Bayes dan IndoBERT digunakan untuk melakukan klasifikasi teks ke dalam empat kategori tingkat kasar. Berikut adalah perbandingan ketiga model tersebut

Aspek	Naive Bayes / Logistic Regression	IndoBERT
Pendekatan	Machine Learning tradisional berbasis probabilitas	Deep learning berbasis Transformer
Input fitur	TF-IDF (representasi numerik frekuensi kata)	Embedding kontekstual dari AutoTokenizer
Pemahaman konteks	Tidak — setiap kata dianggap independen	Ya — memahami hubungan antar kata secara dua arah
Kecepatan pelatihan	Sangat cepat	Lebih lambat, memerlukan GPU
Performa umum	Baik sebagai baseline	Lebih tinggi, terutama untuk kata ambigu

Tabel 2. Perbandingan karakteristik model

3.4.1. Logistic Regression

Logistic Regression dilatih menggunakan representasi TF-IDF dari teks yang telah di *preprocess*. Model ini menghasilkan output berupa label klasifikasi dengan nilai 0 hingga 3 yang menunjukkan tingkat kekasaran. Logistic Regression berfungsi sebagai baseline pertama dan dibandingkan dengan Naive Bayes serta IndoBERT menggunakan metrik F1-score.

3.4.2. Naive Bayes Classifier

Naive Bayes dilatih menggunakan representasi TF-IDF yang sama dengan Logistic Regression. Berdasarkan probabilitas kemunculan fitur dalam setiap kelas, model mengklasifikasikan teks ke dalam salah satu dari empat tingkat kekasaran (0–3). Meskipun asumsi independensi antar fitur menyederhanakan realitas, Naive Bayes umumnya memberikan performa yang kompetitif pada klasifikasi teks dan berfungsi sebagai baseline kedua dalam proyek ini.

3.4.3. IndoBERT

Teks yang telah di *preprocess* diubah menjadi token menggunakan AutoTokenizer dari Hugging Face yang spesifik untuk model IndoBERT. Token yang dihasilkan dikonversi menjadi embedding kontekstual dan diproses oleh model IndoBERT untuk memahami konteks kalimat secara keseluruhan. IndoBERT kemudian di-fine-tune menggunakan dataset yang telah dianotasi

untuk tugas klasifikasi empat kelas (0–3). Output akhir berupa label klasifikasi tingkat kekasaran beserta skor kepercayaan (confidence score).

Keunggulan utama IndoBERT dibanding kedua model tradisional adalah kemampuannya memahami konteks kata. Kata ‘anjing’ dalam kalimat ‘anjing saya sakit’ akan menghasilkan representasi embedding yang berbeda dibanding ‘anjing’ dalam kalimat ‘dasar anjing lu!’, sehingga model dapat membedakan makna netral dan makna kasar dari kata yang sama.

3.5. Evaluasi *Output*

Evaluasi model dilakukan untuk mengukur performa dari kedua model dalam mengklasifikasi tingkat kata kasar pada teks. Dataset yang digunakan untuk data latih dan data uji dengan perbandingan 80:20. Terdapat dua metode evaluasi yang digunakan:

3.5.1. F-1 Score

Metode evaluasi utama yang digunakan adalah F1-Score, karena metrik ini mampu memberikan keseimbangan antara precision dan recall, terutama dalam kasus klasifikasi data yang tidak seimbang (imbalanced). Precision mengukur seberapa banyak prediksi positif yang benar-benar positif, sedangkan recall mengukur seberapa banyak data positif yang berhasil terdeteksi oleh model. F1-Score dihitung untuk setiap kelas (0, 1, 2, 3) dan dilaporkan sebagai macro-average untuk memberikan gambaran keseluruhan performa model.

3.5.1. Confusion Matrix

Confusion matrix digunakan untuk melihat distribusi hasil prediksi model terhadap masing-masing kelas secara detail. Dengan confusion matrix, dapat dianalisis jenis kesalahan klasifikasi yang dilakukan oleh model, misalnya apakah model cenderung mengklasifikasikan teks Level 2 sebagai Level 1 atau sebaliknya. Evaluasi dilakukan pada ketiga model (Logistic Regression, Naive Bayes, dan IndoBERT) untuk membandingkan performa ketiganya secara komprehensif..

Evaluasi dilakukan pada ketiga model (Logistic Regression, Naive Bayes, dan IndoBERT) untuk membandingkan performa keduanya dalam mendeteksi kata kasar. Hasil evaluasi ini digunakan untuk menentukan model mana yang lebih efektif.

DAFTAR PUSTAKA

- Apriliani, D., Handayani, S. F., & Saputra, I. T. (2023). Implementasi Natural Language Processing (NLP) Dalam Pengembangan Aplikasi Chatbot Pada SMK YPE Nusantara Slawi. *Techno.com*, 22(4), 1037-1047. <https://pdfs.semanticscholar.org/07c7/bd164462bd42d18c34cd38bf15472645dfe7.pdf>
- LLC, C. T. (2025). *Natural Language Processing with Python: Master Text Processing, Language Modeling, and NLP Applications with Python's Powerful Tools*. Packt Publishing. Retrieved April 13, 2026, from https://www.google.co.id/books/edition/Natural_Language_Processing_with_Python/EG4_EQAAQBAJ?hl=id&gbpv=0
- Prahasto, G. S., & Setiawan, E. B. (2025). Twitter Social Media-Based Sentiment Analysis Using Bi-LSTM Method With Genetic Algorithms Optimization. *Khazanah Informatika*, 11(1), 20-26. <https://doi.org/10.23917/khif.v11i1.3959>